

| BACHELOR 3 CPI — PARCOURS DÉVELOPPEMENT

Document de Cadrage

Althea Systems — Plateforme e-commerce B2B de matériel médical

ÉQUIPE PROJET — KYSS

Keep Your System Safe

Samy Abdelmalek · Tristan Sanjuan
Rayan Menkar · Kelvin Chauvel

École

Sup de Vinci

Année

2025 — 2026

Promotion

Bachelor 3 — CPI Développement

Table des matières

1. Résumé exécutif

2. Introduction et contexte

- 2.1 Présentation du client
- 2.2 Cadre pédagogique
- 2.3 Objet du présent document

3. Reformulation du besoin client

- 3.1 Expression du besoin
- 3.2 Parties prenantes
- 3.3 Cibles utilisateurs

4. Analyse fonctionnelle

- 4.1 Périmètre global
- 4.2 Priorisation MoSCoW

5. Contraintes du projet

- 5.1 Contraintes réglementaires
- 5.2 Contraintes d'accessibilité
- 5.3 Contraintes d'expérience utilisateur
- 5.4 Contraintes de performance
- 5.5 Contraintes de sécurité

6. Comment la plateforme est construite

- 6.1 Vision générale
- 6.2 Les principales briques
- 6.3 Comment un utilisateur se connecte

6.4 Parcours de commande

7. Choix des outils

- 7.1 Comment on a choisi
- 7.2 Le site web — Next.js
- 7.3 La base de données — PostgreSQL
- 7.4 Le cache — Redis
- 7.5 La connexion et les comptes — NextAuth
- 7.6 Le paiement — Stripe
- 7.8 Le stockage des photos — Cloudflare R2
- 7.9 L'envoi des emails — Resend
- 7.10 L'hébergement du site — serveur privé + Dokploy
- 7.11 Le multilingue — next-intl
- 7.12 Récapitulatif

8. Modèle de données

- 8.1 Vue générale
- 8.2 Principales entités

9. Méthodologie projet

- 9.1 Cadre retenu : une approche agile adaptée
- 9.2 Gestion du code et des tâches
- 9.3 Outils de communication

10. Répartition des rôles

- 10.1 Composition de l'équipe
- 10.2 Matrice des responsabilités
- 10.3 Fiches de rôle

11. Planning prévisionnel

- 11.1 Vue d'ensemble
- 11.2 Jalons certifiants
- 11.3 Dépendances clés

11.4 Représentation graphique du planning

12. Livrables par phase

13. Veille technologique

13.2 Un concurrent sérieux : Remix

13.3 Norme essentielle : le RGPD

14. Gestion des risques

14.1 Identification et évaluation

14.2 Plan de continuité

15. Charte graphique

15.1 Palette de couleurs officielle

15.2 Statuts produits

15.3 Typographie et iconographie

16. Conclusion

17. Annexes

17.1 Annexe A — Documentation complémentaire

17.2 Annexe B — Glossaire

17.3 Annexe C — Maquettes Figma

1 Résumé exécutif

Ce document présente la réponse de l'équipe projet à l'appel d'offre émis par **Althea Systems**, entreprise spécialisée dans la distribution de matériel médical de pointe pour cabinets médicaux. Le client souhaite se doter d'une plateforme e-commerce B2B internationale, mobile-first, sécurisée et accompagnée d'un back-office de gestion autonome.

Le projet est conduit par une équipe de **quatre étudiants de Bachelor 3 CPI parcours Développement** dans le cadre du projet d'étude annuel.

La solution repose sur des outils éprouvés, utilisés par des géants du web comme Netflix ou Amazon. Elle offre des pages rapides sur mobile comme sur ordinateur, un paiement encaissé par Stripe (le prestataire bancaire de référence, utilisé par des millions de boutiques), une double sécurité à la connexion pour les administrateurs, et un site disponible en français, anglais et arabe (avec écriture de droite à gauche).

Le calendrier prévisionnel couvre neuf phases réparties de septembre 2025 à juin 2026, avec les jalons certifiants imposés par le cadre pédagogique Sup de Vinci : kick-off, rendez-vous de cadrage, document de cadrage, soutenance et livrables finaux.

2 Introduction et contexte

2.1 Présentation du client

Althea Systems commercialise du matériel médical haut de gamme destiné aux cabinets médicaux. Historiquement, la distribution s'effectue par un réseau de commerciaux et de distributeurs spécialisés. L'entreprise souhaite aujourd'hui **se transformer numériquement** pour :

- Faciliter l'accès à son catalogue pour les praticiens existants, qui pourront commander 24 heures sur 24.
- Capter de nouveaux marchés internationaux, notamment dans les zones francophones, anglophones et arabophones.
- Moderniser l'image de marque et aligner l'expérience client sur les standards actuels du e-commerce B2B.

- Soulager les équipes commerciales des tâches transactionnelles à faible valeur ajoutée, pour les recentrer sur le conseil.

2.2 Cadre pédagogique

Ce projet constitue le **projet d'étude annuel** de la promotion Bachelor 3 CPI parcours Développement 2025-2026 de Sup de Vinci. Il permet de valider les trois blocs de compétences suivants :

Bloc	Intitulé	Validé par
BC1	Piloter un projet informatique	Rendez-vous de cadrage (20 %) + Document de cadrage (80 %)
BC2	Coordonner une équipe projet	Soutenance (80 %) + Analyse de la dynamique projet (20 %)
BC3	Superviser la mise en œuvre d'un projet	Livrables finaux techniques (100 %)

2.3 Objet du présent document

Le Document de Cadrage a pour vocation de :

- **Reformuler** de manière précise le besoin exprimé par le client.
- **Structurer** la réponse fonctionnelle et technique à l'appel d'offre.
- **Justifier** les choix d'outils retenus au regard de critères objectifs : performance, sécurité, maintenabilité, évolutivité et coût.
- **Planifier** les étapes de réalisation avec des jalons et des livrables clairs.
- **Identifier** les rôles, responsabilités et risques du projet.

Il est rédigé dans une démarche de **recherche et de réflexion**, en évitant l'exhaustivité gratuite : chaque choix est motivé par le cahier des charges du client.

3 Reformulation du besoin client

3.1 Expression du besoin

« Nous souhaitons développer une plateforme e-commerce innovante pour proposer nos produits à une clientèle internationale. Cette plateforme permettra à nos clients d'acheter et de commander facilement nos produits directement depuis leur interface. Nous voulons offrir une expérience d'achat fluide et moderne, en adéquation avec les attentes actuelles du marché. »

— Cahier des charges Althea Systems.

Reformulé en langage projet, le besoin repose sur **quatre piliers** hiérarchisés explicitement par le client :

- **Sécurité** — plateforme et paiement sécurisés, avec une attention particulière aux données bancaires.
- **Expérience utilisateur** — fluide, responsive, mobile-first, cohérente entre desktop et mobile.
- **Back-office complet** — gestion autonome des produits, catégories, commandes, factures et contenus éditoriaux par l'équipe interne d'Althea.
- **Maintenabilité et évolutivité** — capacité à intégrer de nouvelles fonctionnalités à long terme sans remettre en cause les fondations.

3.2 Parties prenantes

Partie prenante	Rôle	Intérêt principal
Althea Systems (direction)	Client / commanditaire	Retour sur investissement, image de marque, nouveaux marchés
Équipes commerciales internes	Utilisateurs du back-office	Gestion produits, suivi des commandes, relation client
Praticiens et cabinets médicaux	Utilisateurs finaux B2B	Rapidité, fiabilité, disponibilité du service
Équipe projet (4 étudiants)	Maîtrise d'œuvre	Livraison, apprentissage, validation RNCP
Encadrement Sup de Vinci	Sponsor académique	Évaluation et conformité au référentiel

3.3 Cibles utilisateurs

Deux profils principaux, avec des exigences différenciées :

- **Le client B2B** (praticien ou acheteur pour un cabinet) : recherche rapide d'un produit, fiche détaillée, processus de commande court, accès simple à son historique et à ses factures.
- **L'administrateur Althea** (gestionnaire catalogue ou service client) : pouvoir créer, modifier, supprimer des produits, suivre l'activité commerciale via un tableau de bord, traiter les messages des clients.

3.4 Objectifs stratégiques et indicateurs de succès

Objectif	Indicateur visé
Disponibilité du service	Taux de disponibilité supérieur à 99,9 % (environ 8 h d'interruption maximum par an)
Réactivité du site	Pages rapides, résultats de recherche quasi instantanés
Confiance	Zéro incident de sécurité majeur, conformité RGPD
Ouverture internationale	Trois langues opérationnelles dès la mise en production
Inclusion	Site accessible aux personnes en situation de handicap (norme WCAG 2.1)

4 Analyse fonctionnelle

4.1 Périmètre global

Le cahier des charges décrit un périmètre large, réparti en trois grandes familles :

- **Frontend public** : page d'accueil, catalogue, recherche, fiche produit, panier, checkout, confirmation, contact et chatbot.
- **Espace client** : inscription, connexion, mot de passe oublié, profil, carnet d'adresses, moyens de paiement, historique des commandes et des factures.
- **Back-office administrateur** : gestion des produits, des catégories, des commandes, des utilisateurs, des factures, des avoirs, des messages, du contenu éditorial de la page d'accueil, et tableaux de bord analytiques.

4.2 Priorisation MoSCoW

La méthode MoSCoW (*Must, Should, Could, Won't*) permet de distinguer l'indispensable du souhaitable. Elle sert de référence à l'équipe pour arbitrer en cas de dérive de planning.

4.2.1 Must have — indispensable à la mise en production

Fonction	Justification
Catalogue navigable (catégories, produit, recherche)	Cœur métier e-commerce
Panier et checkout avec paiement en ligne	Sans paiement, pas de revenus
Authentification sécurisée	Prérequis à toute commande ou suivi
Back-office de gestion des produits, catégories et commandes	Autonomie éditoriale de l'équipe Althea
Facture téléchargeable	Obligation comptable pour le client B2B
Expérience mobile-first	Contrainte explicite du cahier des charges
Double authentification pour les administrateurs	Protection du back-office
Chatbot avec persistance des conversations en base	Support utilisateur + traçabilité exigée par le CDC §XV

4.2.2 Should have — fortement attendu

- Tableau de bord analytique (ventes par jour, par catégorie, panier moyen).
- Gestion des avoirs avec génération PDF.
- Recherche rapide à facettes (titre, description, caractéristiques techniques, prix, catégorie, disponibilité).
- Multilingue (français, anglais, arabe).
- Conformité à l'accessibilité WCAG 2.1 niveau AA.
- Export CSV ou Excel depuis l'administration.
- Carrousel d'accueil éditable avec éditeur de texte riche (gras, italique, liens, couleurs).

4.2.3 Could have — nice to have

- Notifications par email enrichies.
- Liste de souhaits.
- Codes promotionnels.

4.2.4 Won't have — hors périmètre

- Application mobile native (le site mobile-first couvre le besoin).
- Marketplace multi-vendeurs.
- Programme de fidélité à points.
- Paiement en plusieurs fois.

5 Contraintes du projet

5.1 Contraintes réglementaires

Le projet traite des **données personnelles** (identité, adresses, email, historique d'achat) et des **données de paiement**. Deux cadres juridiques et normatifs s'appliquent :

- **Le RGPD** (Règlement Général sur la Protection des Données). Il impose notamment une information préalable des utilisateurs, des droits d'accès, de rectification, d'effacement et de portabilité, ainsi qu'un niveau de sécurité approprié et une politique de conservation des données.
- **La norme PCI-DSS** (sécurité des données bancaires). Les données de carte bancaire n'étant jamais stockées ni manipulées directement par la plateforme — elles sont déléguées à un prestataire certifié —, le périmètre de conformité est grandement réduit. C'est un choix volontaire et structurant.

5.2 Contraintes d'accessibilité

Le cahier des charges demande explicitement le respect de la norme **WCAG 2.1 niveau AA**. Cela signifie que la plateforme doit être utilisable par toute personne, y compris en situation de handicap visuel, moteur ou cognitif. Concrètement :

- Navigation possible entièrement au clavier.
- Contrastes de couleurs suffisants.
- Alternatives textuelles aux images.
- Compatibilité avec les lecteurs d'écran.
- Gestion du focus visible.

Un audit a déjà été engagé sur les pages et composants principaux, avec des corrections appliquées.

5.3 Contraintes d'expérience utilisateur

- **Mobile-first** : le site doit être conçu d'abord pour les écrans mobiles, puis adapté au desktop.
- **Multilingue** : trois langues cibles, le français, l'anglais et l'arabe. L'arabe implique une gestion particulière de l'écriture de droite à gauche.
- **Cohérence visuelle** : respect de la charte graphique fournie par le client (palette teal et navy).

5.4 Contraintes de performance

Le cahier des charges mentionne explicitement une exigence de **résultats de recherche rapides**. Plus largement, la plateforme doit offrir :

- Des pages rapides à s'afficher, y compris sur un mobile en 4G.
- Une recherche fluide, sans latence perceptible.
- Une scalabilité permettant d'absorber une croissance du trafic sans refonte.

5.5 Contraintes de sécurité

Au-delà du RGPD et du PCI-DSS, la plateforme doit se prémunir contre les menaces web classiques (le *top 10* de l'OWASP) : injection, cross-site scripting, usurpation de session, mauvaise configuration, dépendances vulnérables. Les mesures adoptées sont décrites dans la section dédiée à l'architecture.

6 Comment la plateforme est construite

6.1 Vision générale

Plutôt que de construire une usine à gaz avec plein de petits programmes qui se parlent entre eux (difficile à gérer à 4 étudiants), on construit **une seule application bien organisée**, avec des compartiments clairs : un pour les comptes, un pour le catalogue, un pour les commandes, un pour l'administration.

Les avantages :

- **Un seul projet à mettre en ligne** (au lieu d'une dizaine) : on gagne du temps et on limite les risques de panne.
- **Coût d'hébergement réduit** : environ 30 € par mois en production.

- **Développement plus rapide** pour une équipe de 4.
- **Prêt à grandir** : si la plateforme devait gérer 100 fois plus de commandes, on pourrait détacher certains compartiments (par exemple le paiement) sans tout refaire.

6.2 Les principales briques

La plateforme s'appuie sur les éléments suivants :

- **Le site lui-même** : ce que voit le visiteur, l'espace client et le back-office administrateur réunis dans un projet unique.
- **La base de données** : stocke tous les comptes, produits, catégories, commandes, factures, avoirs, adresses et messages.
- **Un système d'accélération** (Redis) : garde en mémoire les informations les plus consultées pour que les pages s'affichent plus vite, et bloque automatiquement les attaques (trop de tentatives en peu de temps).
- **Un prestataire de paiement** (Stripe) : s'occupe d'encaisser l'argent et de gérer la sécurité bancaire.
- **Un prestataire d'emails** (Resend) : envoie les confirmations de commande, les validations d'inscription, les mots de passe oubliés.
- **Un hébergement cloud pour les photos** (Cloudflare R2) : stocke toutes les images produits et les diffuse rapidement partout dans le monde.

6.3 Comment un utilisateur se connecte

L'utilisateur crée un compte avec son email et un mot de passe, ou se connecte avec son compte Google / GitHub (plus rapide, un clic). Pour les administrateurs, on ajoute une deuxième sécurité : un code à 6 chiffres qui change toutes les 30 secondes, généré par une appli mobile (Google Authenticator par exemple). Même si un pirate volait le mot de passe, il ne pourrait pas entrer sans le téléphone du vrai administrateur.

6.4 Parcours de commande

Le parcours est organisé en quatre étapes successives :

1. **Authentification** (ou poursuite en tant qu'invité).
2. **Saisie de l'adresse** de facturation et, si nécessaire, de livraison.
3. **Paiement** : l'utilisateur saisit sa carte dans un formulaire dédié du prestataire de paiement, sans que la plateforme ne voie jamais le numéro de carte.

4. **Confirmation** : la commande est enregistrée, la facture générée automatiquement au format PDF, un email de confirmation est envoyé au client.

Un mécanisme de notification automatique en provenance du prestataire de paiement permet à la plateforme d'être informée en temps réel du succès ou de l'échec du paiement, garantissant la cohérence entre ce qui a été encaissé et ce qui est enregistré.

7 Choix des outils

7.1 Comment on a choisi

Pour chaque brique de la plateforme, on s'est posé la même question : **quel outil permet d'aller plus vite, coûte moins cher, est plus sûr et plus simple à maintenir ?**

Chaque choix est comparé à une ou deux alternatives, avec une justification concrète. Pas de préférence personnelle : on prend ce qui répond le mieux au cahier des charges.

7.2 Le site web — Next.js

Ce qu'on a pris : Next.js.

Pourquoi : c'est l'outil qui permet de construire **tout le site en un seul projet** (l'affichage des pages + les traitements derrière), au lieu d'avoir deux projets séparés à gérer. Résultat : moins de code, moins d'erreurs possibles, développement plus rapide. Les pages s'affichent aussi plus vite car elles sont préparées côté serveur avant d'être envoyées au navigateur.

C'est utilisé par : Netflix, TikTok, Nike, Hulu, Notion, OpenAI — des géants du web qui ont besoin de sites rapides et fiables.

Alternative rejetée : faire deux projets séparés (un pour l'affichage, un pour les traitements). Plus long à développer pour une équipe de 4 étudiants, plus de bugs au final.

7.3 La base de données — PostgreSQL

Ce qu'on a pris : PostgreSQL.

Pourquoi : les données d'un site de vente sont **très reliées entre elles** (une commande est liée à un client, qui a une adresse, qui a une carte, qui a des produits dans son panier). PostgreSQL est conçu pour gérer ce genre de liens sans erreur. Il est gratuit, utilisé partout depuis 25 ans, extrêmement fiable.

Alternative rejetée : MongoDB. Conçue pour des données sans liens entre elles (genre des posts indépendants). Sur un e-commerce, ça créerait des incohérences (commandes sans client, factures sans commande...). Inadapté.

7.4 Le cache — Redis

Ce qu'on a pris : Redis.

Pourquoi : certaines informations sont demandées **des milliers de fois par jour** (le catalogue, les fiches produits). Au lieu d'aller chercher la même info encore et encore dans la base de données, Redis les garde en mémoire rapide. Résultat : pages qui s'affichent **instantanément** au lieu de mettre 1-2 secondes. Redis sert aussi à **empêcher les attaques** (quelqu'un qui essaie de se connecter 1000 fois avec des mots de passe différents) en limitant automatiquement le nombre de requêtes.

7.5 La connexion et les comptes — NextAuth

Ce qu'on a pris : NextAuth.

Pourquoi : gérer les inscriptions, connexions, mots de passe oubliés, c'est long et risqué à développer à la main (risques de failles de sécurité). NextAuth est **gratuit** et permet de tout faire en quelques lignes : connexion par email, connexion avec son compte Google ou GitHub (plus rapide pour l'utilisateur).

Alternative rejetée : des services payants comme Auth0 ou Clerk. Ils font la même chose mais à partir de 23 \$/mois, ce qui est disproportionné pour un projet étudiant.

Double sécurité pour les admins : en plus du mot de passe, on demande un code à 6 chiffres généré par une appli mobile type Google Authenticator. Si un hacker vole le mot de passe, il ne peut pas rentrer sans ce code.

7.6 Le paiement — Stripe

Ce qu'on a pris : Stripe.

Pourquoi : traiter des cartes bancaires est **hyper régulé** (normes PCI-DSS). Stripe est le plus haut niveau de certification bancaire au monde. L'utilisateur tape sa carte dans un formulaire Stripe qu'on intègre à notre site, mais **nos serveurs ne voient jamais le numéro de carte**. Résultat : zéro responsabilité juridique pour nous en cas de fuite.

C'est utilisé par : Amazon, Google, Shopify, Apple Pay, Airbnb, Uber, Zoom...

Alternative rejetée : PayPal. Très connu aussi mais plus cher en commission, moins bien intégré aux sites modernes.

7.7 L'apparence du site — Tailwind, shadcn et Tiptap

Ce qu'on a pris : Tailwind CSS, shadcn/ui, Tiptap.

Pourquoi : - **Tailwind** permet de construire le design **deux fois plus vite** qu'en écrivant du CSS classique à la main. On assemble des briques déjà prêtes. - **shadcn/ui** fournit des boutons, menus, tableaux **déjà accessibles** aux personnes handicapées (lecteurs d'écran, navigation clavier). Ça nous fait gagner des semaines de travail sur l'accessibilité. - **Tiptap** permet aux administrateurs d'écrire du texte avec **gras, italique, liens, couleurs** (comme dans Word) dans le back-office, sans qu'ils aient à connaître le code. Exigé par le cahier des charges pour le carrousel de la page d'accueil.

7.8 Le stockage des photos — Cloudflare R2

Ce qu'on a pris : Cloudflare R2.

Pourquoi : héberger des milliers de photos produits coûte cher chez les géants classiques (Amazon S3 facture à chaque fois qu'une image est téléchargée). Cloudflare R2 fait la même chose **sans frais de téléchargement**. Économie estimée : des dizaines d'euros par mois quand le catalogue grossit. Les photos sont aussi distribuées sur des serveurs répartis dans le monde entier : un client au Maroc reçoit l'image depuis un serveur proche, pas depuis la France. Pages plus rapides.

7.9 L'envoi des emails — Resend

Ce qu'on a pris : Resend.

Pourquoi : envoyer des emails (confirmation de commande, mot de passe oublié) depuis son propre serveur c'est presque toujours bloqué par Gmail/Outlook qui les classent en spam. Resend est un **service spécialisé** qui s'occupe de faire passer les

emails correctement dans les boîtes de réception. Gratuit jusqu'à 3 000 emails par mois, ce qui couvre largement le démarrage.

7.10 L'hébergement du site — serveur privé + Dokploy

Ce qu'on a pris : un serveur privé loué (environ 10 €/mois) avec un outil appelé Dokploy pour le piloter.

Pourquoi : on a le contrôle total (sécurité, données RGPD, hébergement européen), le coût est fixe et prévisible, et Dokploy automatise les mises à jour et les sauvegardes. Le certificat de sécurité HTTPS est renouvelé tout seul.

Alternative rejetée : hébergement chez Vercel (les créateurs de Next.js). Très pratique mais devient vite cher dès que le trafic augmente (70 \$/mois et +), et les données partent aux États-Unis.

7.11 Le multilingue — next-intl

Ce qu'on a pris : next-intl.

Pourquoi : c'est l'outil qui permet de passer le site en **français, anglais ou arabe** en un clic. Gère aussi automatiquement l'écriture **de droite à gauche** pour l'arabe (caractères inversés, menu à droite, etc.). Indispensable pour la clientèle internationale ciblée par Althea Systems.

7.12 Récapitulatif

À quoi ça sert	Outil retenu
Construire le site (affichage + traitements)	Next.js
Stocker les données (comptes, commandes, produits)	PostgreSQL
Accélérer les pages et protéger des attaques	Redis
Gérer les inscriptions et connexions	NextAuth
Double sécurité pour les administrateurs	Application mobile (Google Authenticator)
Encaisser les paiements	Stripe
Mettre en forme le site	Tailwind + shadcn + Tiptap
Héberger les photos produits	Cloudflare R2
Envoyer les emails (confirmation, mot de passe oublié)	Resend
Proposer le site en 3 langues (FR/EN/AR)	next-intl
Héberger le site en production	Serveur privé européen + Dokploy

8 Modèle de données

8.1 Vue générale

La base de données est structurée autour de **six grands domaines**, eux-mêmes composés d'entités reliées entre elles. Elle comporte **dix-sept modèles** et sept énumérations.

- **Identité et authentification** : les comptes utilisateurs, les sessions, les tokens de validation d'email, les codes de secours pour la double authentification.
- **Catalogue** : les catégories (organisables en arborescence) et les produits, ainsi que les éléments éditoriaux de la page d'accueil (slides du carrousel).

- **Commande et facturation** : les commandes, les lignes de commande, l'historique des statuts, les factures et les avoirs.
- **Carnet d'adresses** : les adresses de facturation et de livraison enregistrées par les utilisateurs.
- **Relation client** : les messages issus du formulaire de contact, ainsi que les conversations du chatbot (deux modèles dédiés : une conversation + ses messages) pour assurer la traçabilité et le suivi par les administrateurs, conformément au CDC §XV.

8.2 Principales entités

- **Utilisateur** : identifié par son email, avec un rôle (client ou administrateur), un statut (actif, suspendu, en attente de validation) et des paramètres de sécurité (mot de passe chiffré, double authentification).
- **Produit** : nom, description, caractéristiques techniques, prix, taux de TVA, niveau de stock, statut (disponible, rupture, retiré), images associées, catégorie de rattachement, éventuelle mise en avant.
- **Commande** : client, adresse, liste d'articles achetés, montant total, statut (en attente, confirmée, expédiée, livrée, annulée), statut de paiement (en attente, payée, échouée, remboursée). Une commande conserve un **instantané des prix** au moment de l'achat, pour être indépendante des évolutions futures du catalogue.
- **Facture et avoir** : documents PDF générés automatiquement, rattachés à une commande, numérotés selon une convention interne.

Le schéma relationnel complet (diagramme entité-relation) est fourni en annexe.

9 Méthodologie projet

9.1 Cadre retenu : une approche agile adaptée

L'équipe retient une **démarche agile de type Scrum**, simplifiée pour correspondre à la réalité d'une équipe de quatre personnes étudiantes :

- **Sprints de deux semaines**, avec un périmètre défini en début de sprint.
- **Revue hebdomadaire** informelle et **rétrospective** en fin de sprint.
- **Daily asynchrone** sur Discord pour partager l'avancement sans bloquer.
- **Revue de code obligatoire** par un pair avant tout apport au code principal.

9.2 Gestion du code et des tâches

- **Git et GitHub** pour le versionnement du code.
- **GitHub Projects** pour le suivi des tâches (tableau Kanban : à faire, en cours, en revue, terminé).
- **Pull requests** systématiques, avec intégration continue qui vérifie la qualité du code (vérification de syntaxe, typage, validité de la base de données, construction du projet) avant toute fusion.

9.3 Outils de communication

- **Discord** pour l'échange quotidien, avec des canaux dédiés (général, développement, design, alertes).
- **Figma** pour les maquettes et la charte graphique.
- **Markdown dans le dépôt Git** pour la documentation technique.

9.4 Définitions de *prêt* et *terminé*

Une fonctionnalité est considérée comme **prête à être développée** lorsque ses critères d'acceptation sont rédigés, que la maquette est disponible s'il s'agit d'interface, et que les dépendances éventuelles sont levées.

Une fonctionnalité est considérée comme **terminée** lorsqu'elle est fusionnée dans le code principal, testée, relue par un pair, documentée si elle modifie l'API, et validée sur l'environnement de pré-production.

10 Répartition des rôles

10.1 Composition de l'équipe

L'équipe est composée de quatre étudiants du Bachelor 3 CPI parcours Développement. Chacun prend en charge un domaine principal, tout en contribuant de manière transverse via les revues de code, les tests croisés et les rituels agiles.

Remarque sur la taille d'équipe : le cadre pédagogique mentionne une équipe de deux à trois étudiants. Ce projet mobilise quatre membres, afin de couvrir la largeur fonctionnelle du cahier des charges (site public, back-office, authentification, paiement, internationalisation, accessibilité) dans les délais impartis.

10.2 Matrice des responsabilités

Pour chaque grand domaine, la matrice ci-dessous indique qui est **responsable** (pilote et fait), **consulté** (donne son avis) ou **informé** (tenu au courant). La lettre R désigne le responsable principal de chaque ligne.

Domaine	Samy	Tristan	Rayan	Kelvin
Authentification et double facteur	R			
Infrastructure, hébergement, déploiement	R			
Intégration continue et sécurité transverse	R			
Pages publiques (accueil, catalogue, produit, checkout)		R		
Design system et charte graphique		R		
Accessibilité et responsive		R		
Back-office administrateur (tableaux, dashboard, gestion)			R	
Carrousel, produits mis en avant, exports CSV			R	
API et logique métier				R
Base de données et modélisation				R

Domaine	Samy	Tristan	Rayan	Kelvin
Intégration du paiement				R
Génération des factures et avoirs				R
Multilingue				R

10.3 Fiches de rôle

10.3.1 Samy Abdelmalek — Authentification et infrastructure

- Mise en place de l'authentification (email / mot de passe, Google, GitHub).
- Implémentation de la double authentification pour les administrateurs, avec les codes de secours associés.
- Dockerisation de l'application et déploiement sur serveur privé.
- Configuration du cache, de la limitation de requêtes et du stockage d'images.
- Mise en place de l'intégration continue (pipeline automatique de vérification du code à chaque modification).
- Politique de sécurité globale : en-têtes de sécurité, audits de dépendances.

10.3.2 Tristan Sanjuan — Frontend public et design system

- Développement des pages publiques : accueil (carrousel, grille catégories, top produits), page catégorie, fiche produit, panier, parcours de commande.
- Mise en œuvre de la charte graphique Althea (palette teal et navy).
- Conformité responsive sur toutes les tailles d'écran (mobile à desktop).
- Conformité accessibilité : navigation clavier, contrastes, libellés explicites, gestion du focus.
- Animations et micro-interactions.

10.3.3 Rayan Menkar — Back-office administrateur

- Pages d'administration : tableau de bord, gestion des produits, catégories, utilisateurs, commandes, factures, avoirs, messages.
- Composants de tableau réutilisables avec tri, pagination et actions groupées.
- Exports CSV et Excel des données.
- Éditeur du carrousel d'accueil et gestion des produits vedettes.

- Écran de suivi des échanges chatbot.

10.3.4 Kelvin Chauvel — Backend et intégrations

- Conception et développement de l'API (points d'entrée exposés par la plateforme).
- Modélisation de la base de données et gestion des migrations.
- Intégration du paiement : création des sessions de commande, réception des notifications de paiement, mise à jour des commandes.
- Génération automatique des factures et avoirs au format PDF.
- Envoi des emails transactionnels (confirmation, réinitialisation).
- Mise en place du multilingue.

11 Planning prévisionnel

11.1 Vue d'ensemble

Le projet s'étale sur **neuf mois**, de septembre 2025 à juin 2026, en neuf phases successives (certaines peuvent se recouvrir).

Phase	Période	Durée	Livrable principal
Kick-off et cadrage	Septembre-octobre 2025	4 semaines	Maquettes et backlog initial
Authentification et sécurité	Octobre-novembre 2025	2-3 semaines	Inscription, connexion, double authentification
Back-office contenu	Novembre-décembre 2025	3 semaines	Gestion produits et catégories
Frontend public et recherche	Décembre 2025-janvier 2026	4 semaines	Catalogue, fiche produit, recherche
Checkout et paiement	Janvier-février 2026	3 semaines	Paiement complet, facture PDF, email
Back-office avancé	Février-mars 2026	3 semaines	Dashboard, avoirs, exports
Espace client	Mars 2026	1 semaine	Profil, adresses, historique
Contact et chatbot	Mars-avril 2026	2 semaines	Formulaire et chatbot
Multilingue et accessibilité	Avril 2026	2 semaines	Français, anglais, arabe, WCAG
Tests, documentation, maintenance	Mai-juin 2026	4 semaines	Tests, Swagger, document technique

11.2 Jalons certifiants

Les jalons imposés par le cadre pédagogique sont les suivants :

Jalon	Période	Livrable
Kick-off	Septembre 2025	Constitution de l'équipe, prise en main du sujet
Rendez-vous de cadrage	Kick-off + 2 mois	Entretien client de 15 à 20 minutes
Document de cadrage	Rendez-vous + 3 mois	Le présent document
Soutenance et démonstration	Cadrage + 1 mois	Présentation orale de 60 minutes
Livrables finaux techniques	Soutenance + 2 semaines	Code source et documentation technique
Analyse de la dynamique projet	Livrables + 2 semaines	Document individuel de 2 à 4 pages

11.3 Dépendances clés

- La **phase d'authentification bloque tout le reste** : sans compte utilisateur fonctionnel, impossible de passer commande ou d'accéder à l'administration.
- Le **back-office de contenu précède le frontend public** : il faut pouvoir créer des produits pour pouvoir les afficher.
- Le **checkout dépend du frontend public** : la finalisation de commande s'appuie sur le panier déjà constitué.
- Le **multilingue est volontairement positionné en fin de cycle** pour éviter de devoir retraduire des textes modifiés en cours de route.

11.4 Représentation graphique du planning

[illegible]

12 Livrables par phase

Phase	Livrables
Cadrage	Document de cadrage PDF, maquettes Figma, backlog initial
Authentification et sécurité	Pages d'inscription, connexion, mot de passe oublié, double authentification admin
Back-office contenu	CRUD produits et catégories, upload d'images, actions groupées, export CSV
Frontend public	Accueil, catalogue, fiche produit, recherche, panier persisté
Checkout	Parcours de commande, paiement en ligne, notification automatique, facture PDF, email de confirmation
Back-office avancé	Tableau de bord, graphiques, gestion des commandes et utilisateurs, avoirs PDF
Espace client	Profil, adresses, moyens de paiement, historique des commandes
Contact et chatbot	Formulaire de contact, chatbot widget, escalade vers support humain
Multilingue et accessibilité	Trois langues, écriture de droite à gauche, audit accessibilité validé
Tests et documentation	Tests automatisés, documentation API, document de conception technique

13 Veille technologique

Conformément au cadre pédagogique, la veille est volontairement **ciblée sur trois axes** et non exhaustive.

13.1 Une technologie clé : les outils de développement web nouvelle génération

Depuis 2020, de nouveaux outils permettent de construire des sites qui s'affichent **beaucoup plus vite**, sont mieux classés sur Google, et se développent en deux fois moins de temps qu'avant. Next.js (qu'on a retenu) est le leader de cette nouvelle génération.

Ce que ça change concrètement pour Althea :

- Un client qui ouvre le site sur mobile en 4G voit la page **en moins d'une seconde**, au lieu de 3-4 secondes avec les anciens outils. Un site plus rapide, c'est des taux d'achat plus élevés (étude Google : +7 % de conversions pour chaque seconde gagnée).
- Le site est **bien mieux référencé sur Google** parce que les pages sont livrées déjà construites, au lieu d'être assemblées par le navigateur au dernier moment.
- L'équipe travaille avec **un seul outil** au lieu de plusieurs qui doivent se parler, donc moins de bugs et un développement plus rapide.
- **Utilisé par des géants** : Netflix, TikTok, Nike, Notion, OpenAI. Gage de fiabilité et de pérennité.

13.2 Un concurrent sérieux : Remix

Puisque le cahier des charges impose de construire le site avec un outil de développement, on a comparé Next.js à son principal concurrent direct : **Remix**. Les deux font la même famille de choses (afficher des pages, gérer les traitements, router les URL, communiquer avec la base de données), mais avec des choix techniques différents.

Petit détail amusant : Remix est développé par l'équipe de **Shopify**, qui l'a racheté en 2022. C'est donc un outil robuste, financé par un des géants du e-commerce.

Comparaison	Next.js (retenu)	Remix
Taille de la communauté	130 000+ étoiles GitHub, leader incontesté	30 000+ étoiles, en forte croissance
Entreprises qui l'utilisent	Netflix, TikTok, Nike, Notion, OpenAI, Hulu	Shopify, certaines pages de GitHub, quelques startups
Documentation et tutoriels	Très abondante (blogs, vidéos, formations)	Correcte mais moins fournie
Écosystème de modules additionnels	Énorme (intégrations clé-en-main pour presque tout)	Plus limité, souvent à coder soi-même
Déploiement facile	Fonctionne partout (Vercel, serveur classique, Docker)	Fonctionne partout mais moins optimisé hors Cloudflare
Optimisation automatique des images	Intégrée et très poussée	À ajouter à la main
Rendu côté serveur (pages rapides)	Oui, plusieurs modes au choix	Oui, mais un seul mode disponible

Pourquoi on n'a pas pris Remix :

- **Plus de ressources pour apprendre** sur Next.js : pour une équipe de 4 étudiants, avoir accès à des milliers de tutos et d'exemples est un vrai accélérateur. Sur Remix, on aurait souvent dû débbugger seuls.
- **Intégrations toutes faites** : se connecter à Stripe, à un système d'envoi d'emails, à une base de données, c'est en 10 lignes avec Next.js. Sur Remix, plus de configuration manuelle.
- **Optimisations automatiques** : Next.js fait tout seul la compression des images, la découpe du code pour charger uniquement ce qu'il faut. Sur Remix, il faut s'en occuper soi-même.
- **Recrutement** : si Althea Systems reprend le projet plus tard, trouver un développeur qui connaît Next.js est beaucoup plus facile que trouver un développeur Remix.

Remix reste un excellent outil, et l'écart entre les deux se réduit chaque année. Mais aujourd'hui, pour un projet pédagogique livré en 9 mois par une équipe junior, Next.js offre un meilleur rapport vitesse d'apprentissage / richesse des

fonctionnalités.

13.3 Norme essentielle : le RGPD

Le **Règlement Général sur la Protection des Données** est entré en application en mai 2018. Il s'applique à toute entité traitant les données personnelles de résidents européens. Les sanctions peuvent atteindre 4 % du chiffre d'affaires mondial ou 20 millions d'euros.

Pour Althea, cette norme structure directement plusieurs choix de conception :

- **Information préalable** : une page de politique de confidentialité et une bannière de cookies à trois niveaux (essentiels, analytiques, marketing).
- **Droit d'accès** : l'utilisateur peut exporter toutes ses données dans un fichier téléchargeable.
- **Droit à l'effacement** : l'utilisateur peut supprimer son compte, avec conservation des seules factures (obligation comptable de 10 ans).
- **Minimisation** : seules les données strictement nécessaires sont demandées.
- **Sécurité** : chiffrement des communications, mots de passe protégés, double authentification pour les administrateurs.
- **Sous-traitants** : chaque prestataire externe (Stripe, Resend, Cloudflare) fait l'objet d'un contrat spécifique de traitement de données.

Le RGPD n'est pas seulement une contrainte légale : il structure une **bonne hygiène sécurité** qui bénéficie à tous les utilisateurs.

14 Gestion des risques

14.1 Identification et évaluation

La probabilité (*Faible, Moyenne, Élevée*) et l'impact (1 mineur à 4 critique) sont estimés pour chaque risque, et une réponse est proposée.

Risque	Probabilité	Impact	Réponse
Fuite de données personnelles	Faible	4	Audits réguliers, chiffrement, double authentification, minimisation
Panne du paiement (notifications non reçues)	Moyenne	3	Mécanisme de réessai automatique, journalisation, alertes
Dérive de planning sur une phase critique	Moyenne	3	Sprints courts, rétrospectives, arbitrage en faveur du <i>must have</i>
Indisponibilité d'un membre de l'équipe	Faible	3	Documentation continue, polyvalence, pair programming ponctuel
Performance recherche insuffisante	Moyenne	2	Cache dédié, index de recherche, mesures régulières
Non-conformité accessibilité détectée tardivement	Moyenne	2	Audit intermédiaire, tests automatisés
Bug critique en production	Faible	4	Procédure de retour en arrière rapide, sauvegarde quotidienne

Risque	Probabilité	Impact	Réponse
Dépendance tierce abandonnée	Moyenne	2	Veille mensuelle, audits de dépendances
Coût d'infrastructure imprévu	Faible	2	Budget fixe connu à l'avance
Tentative d'attaque (force brute, déni de service)	Moyenne	3	Limitation de requêtes, protection en amont (Cloudflare)

14.2 Plan de continuité

- **Objectif de reprise après incident** : remise en service en moins de 30 minutes.
- **Objectif de perte de données maximum** : moins de 24 heures (sauvegarde automatique quotidienne).
- **Retour en arrière** possible en quelques minutes grâce à l'outil de déploiement, qui permet de repasser à la version précédente.
- **Post-mortem** systématique en cas d'incident majeur, archivé dans la documentation du projet.

15 Charte graphique

15.1 Palette de couleurs officielle

La charte transmise par le client repose sur une palette sobre, dominée par le teal et le bleu nuit, avec des codes couleurs universels pour les statuts (vert, rouge, orange).

Usage	Couleur
Boutons d'action, liens, badges	Teal #00a8b5
Survol de boutons et liens	Teal clair #33bfc9
Fonds doux d'arrière-plan	Teal très clair #d4f4f7
Titres, navigation, pied de page	Navy profond #003d5c
Statut « en stock », validation	Vert #10b981
Statut « rupture », erreurs	Rouge #ef4444
Statut « stock faible », alertes	Orange #F59E0B

15.2 Statuts produits

Quatre badges sont utilisés pour indiquer l'état d'un produit : **en stock** (vert), **stock faible** (orange), **rupture** (rouge), **nouveau** (teal).

15.3 Typographie et iconographie

- Typographie sans serif moderne, lisible à toutes les tailles d'écran.
- Hiérarchie claire entre titres et corps de texte.
- Iconographie cohérente issue de la bibliothèque Lucide (intégrée à shadcn/ui), en ligne fine.

16 Conclusion

Ce document de cadrage pose les bases d'un projet e-commerce **ambitieux mais maîtrisable**, répondant précisément aux exigences du cahier des charges d'Althea Systems, et structuré pour être mené à terme par une équipe de quatre étudiants sur neuf mois.

Les grands choix — architecture simple et robuste, framework web moderne, délégation du paiement à un prestataire certifié, internationalisation dès la conception, accessibilité intégrée, double authentification pour les administrateurs — ont été motivés par des critères clairs : valeur pour le client, sécurité, performance, maintenabilité, coût.

Les **neuf phases du projet** sont identifiées, avec des livrables précis, une **répartition des rôles** claire, une **matrice de risques** accompagnée de plans de réponse, et une **veille technologique** ciblée qui valide la pertinence de la solution au regard du marché.

L'équipe reste à disposition du client Althea Systems et de l'encadrement pédagogique pour tout approfondissement, ajustement du périmètre ou arbitrage en cours de réalisation.

17 Annexes

17.1 Annexe A — Documentation complémentaire

Le dépôt Git du projet contient la documentation suivante, qui complète le présent document :

Document	Rôle
Architecture	Vue d'ensemble technique
Document de conception technique	Livable technique final
Diagrammes techniques	Schémas exportables
Documentation API	Points d'entrée exposés
Procédure de déploiement	Mise en production
Procédure d'installation	Développement local
Politique de maintenance	Sauvegardes, mises à jour, mise à l'échelle
Audit accessibilité	Conformité WCAG 2.1
Audit responsive	Tests multi-écrans
Feuille de route	Découpage par phases
Rapport de sécurité	Vérifications automatisées
Audit des tickets	Suivi des issues GitHub

17.2 Annexe B — Glossaire

Terme	Définition
MoSCoW	Méthode de priorisation des fonctionnalités en quatre catégories : indispensable, fortement attendu, souhaité, exclu
SSR	Rendu d'une page côté serveur plutôt que côté navigateur, pour un affichage plus rapide et un meilleur référencement
Double authentification	Mécanisme de sécurité qui ajoute une deuxième preuve d'identité à la connexion (code à six chiffres)
PCI-DSS	Norme internationale de sécurité pour les paiements par carte bancaire
RGPD	Règlement européen de protection des données personnelles
WCAG	Recommandations internationales d'accessibilité web
RTL	Écriture de droite à gauche, utilisée notamment pour l'arabe
Backlog	Liste ordonnée des fonctionnalités à développer
Sprint	Itération de travail courte (deux semaines ici), avec un périmètre défini
Revue de code	Relecture d'un apport au code par un pair avant intégration
Intégration continue	Vérification automatique de la qualité du code à chaque modification

17.3 Annexe C — Maquettes Figma

Les maquettes haute-fidélité du site public et du back-office administrateur sont consultables sur Figma :

<https://www.figma.com/make/82inzmlnbln7p0VHPEqZVp/Créer-maquettes-de-pages-web>

Ces maquettes couvrent l'ensemble des parcours utilisateurs décrits dans le cahier des charges (accueil, catalogue, fiche produit, panier, tunnel de commande, espace client, back-office) et servent de référence visuelle pour l'équipe tout au long du développement.

Fin du Document de Cadrage — Althea Systems.